GUIA DO ALUNO

Sensor de distância: do eco do som ao alarme que avisa pelo celular

HC-SR04, porta serial, Python ao vivo, buzzer, LED RGB, ESP32, Wi-Fi, WhatsApp e e-mail

---

Placas: Arduino Uno → ESP32 • **Componentes:** HC-SR04, buzzer MH-FMD, LED RGB, resistores

Projetos: gráfico ao vivo em Python • alarme de ré • monitor web local • alerta pela internet

Carga horária: 6 aulas de 100 min

Espaço Cria — 2026

Sumário

Como este curso funciona

Segurança e cuidados

Aula 1 — Medindo distância com som

Aula 2 — Python lendo a serial e plotando ao vivo

Aula 3 — Buzzer: fazendo som

Aula 4 — Mini projeto: alarme de ré

Aula 5 — ESP32: monitor web no celular

Aula 6 — Alerta pela internet: WhatsApp e e-mail

Para ir além

Glossário

Anexo — Ficha de medições (Aula 1)

Anexo — Checklist

Como este curso funciona

Vamos pegar um sensor barato — o **HC-SR04**, que mede distância com som — e construir, aula a aula, um sistema completo: medir, filtrar, enviar para o computador, desenhar um gráfico ao vivo, avisar com som e luz, e por fim mandar um alerta pelo Wi-Fi para o celular e pela internet. Em cada etapa há **física** (som, eco, velocidade, tensão), **lógica** (estados, tempo, filtros, protocolos) e **programação** (C++ no Arduino, Python no PC, HTML/JS na página web).

Aula	Tema	Você vai construir	Sketches / scripts
1	HC-SR04 e porta serial	Um medidor de distância que imprime no Monitor Serial	01, 01b
2	Python lê a serial	Gráfico distância × tempo ao vivo no PC	ler_serial.py, plot_distancia.py
3	Buzzer e melodias	Bipes, sirene, escala e uma música	02, 02b
4	Mini projeto: alarme de ré	Sensor + LED RGB + buzzer, como num caminhão	03
5	ESP32 + monitor web	Página no celular com gráfico e alarme	04
6	Alerta pela internet	WhatsApp e e-mail quando algo se aproxima	05

Como usar este guia

Cada aula: **física/ideia → materiais → montagem (figura) → código → entenda → experimente → desafios**. Os desafios são de lógica: montagem simples, raciocínio caprichado.

Códigos em `aula-distancia/codigo/` (Arduino) e `aula-distancia/python/` (Python). Todos testados.

Segurança e cuidados

- O HC-SR04 funciona com **5 V**. No ESP32 (3,3 V) o pino ECHO precisa de um **divisor de tensão** — sem ele você pode queimar o pino do ESP32 (Aula 5).
- LEDs sempre com resistor. Buzzer passivo direto no pino (ele consome pouco).
- Desligue o USB antes de mexer nos fios.
- O ultrassom de 40 kHz é inaudível e inofensivo, mas não aponte o sensor para o ouvido de ninguém de perto: os transdutores são delicados.
- Senhas de Wi-Fi, chaves de API e senhas de app (Aula 6) são **pessoais**: não compartilhe nem deixe num código público.

Aula 1 — Medindo distância com som

O que você vai aprender

- Como um morcego (e o HC-SR04) mede distância: tempo do eco.
- A física: velocidade do som, ida e volta, e a **regra de três** que transforma microssegundos em centímetros.
- `pulseIn()`, `delayMicroseconds()` e a porta serial como "cano de texto" entre o Arduino e o PC.
- Por que filtrar leituras e o que é uma **mediana**.

Materiais

- 1 Arduino Uno + cabo USB
- 1 protoboard
- 1 sensor HC-SR04
- 4 jumpers (VCC, GND, TRIG, ECHO)
- Régua ou trena para conferir as medidas

A física: eco e velocidade do som

O sensor tem dois "olhos": um **emissor** (T) e um **receptor** (R). Quando o Arduino manda um pulso de 10 µs no pino TRIG, o emissor solta 8 cliques de ultrassom a **40 kHz** (o ouvido humano só vai até ~20 kHz). O som viaja, bate no objeto e volta. O pino **ECHO** fica em nível alto exatamente durante o tempo entre a emissão e o retorno do eco. O Arduino mede esse tempo com `pulseIn()`.

O som no ar a 20 °C anda a **343 m/s**. Em unidades do Arduino: 343 m/s = 34 300 cm/s = **0,0343 cm por microssegundo**. Mas atenção: o tempo medido é de **ida e volta**. Então:

Passo	Conta	Resultado
Distância total percorrida pelo som	tempo (µs) × 0,0343 cm/µs	ida + volta
Distância até o objeto	÷ 2	só a ida
Fórmula final	d (cm) = tempo (µs) × 0,0343 ÷ 2	= tempo ÷ 58,3

Essa última linha é uma **regra de três**: se 58,3 µs correspondem a 1 cm, então *t* µs correspondem a *t* ÷ 58,3 cm. Exemplo: eco em 1166 µs → 1166 ÷ 58,3 = **20 cm**. É assim que uma medida **física** (um tempo) vira um **número digital** (centímetros) dentro do programa.

Alcance e limites do HC-SR04

Mede de **2 cm a ~400 cm**, com cone de ~15°. Objetos macios (tecido, espuma) absorvem o som e "somem"; superfícies inclinadas desviam o eco. Se o eco não volta em 30 ms (~5 m), `pulseIn` devolve 0 — tratamos como "fora de alcance".

A velocidade do som muda com a **temperatura**: v ≈ 331 + 0,6·T (m/s). A 35 °C é 352 m/s — 2,6 % a mais. Um sensor de temperatura corrigiria isso (desafio 1.5).

Montagem

Pino do HC-SR04	Arduino
VCC	5V
GND	GND
TRIG	pino 9
ECHO	pino 10

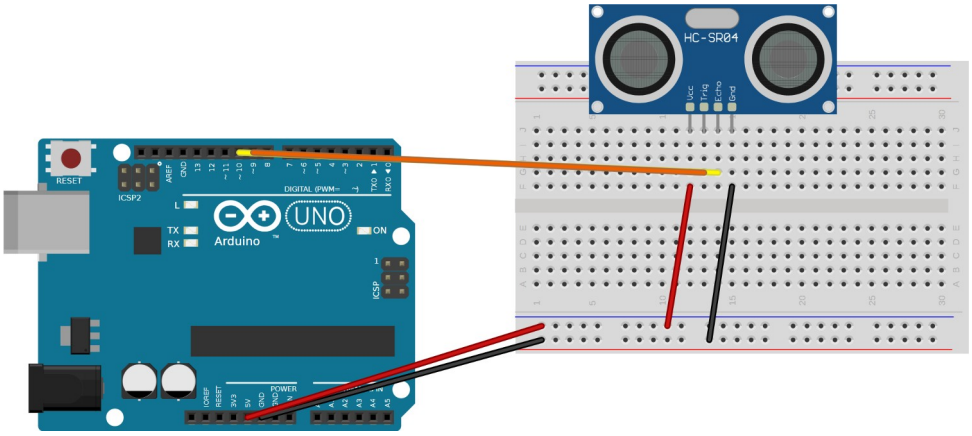


Figura 1 — Aula 1 — HC-SR04 no Arduino Uno: alimentação pelas trilhas, TRIG no pino 9 e ECHO no pino 10.

Código 1 — leitura básica

01_hcsr04_basico.ino

```
/*
  01 - HC-SR04: medindo distancia com som
  Placa: Arduino Uno/Nano

  Ligacoes do HC-SR04:
    VCC  -> 5V        GND  -> GND
    TRIG -> pino 9    ECHO -> pino 10

  Como funciona (fisica):
    1. Mandamos um pulso de 10 us no TRIG.
    2. O sensor emite 8 "cliques" ultrassonicos de 40 kHz (inaudiveis).
    3. O pino ECHO fica em HIGH ate o eco voltar. pulseIn() mede esse tempo em microssegundos.
    4. O som anda a ~343 m/s = 0,0343 cm/us. O tempo medido e de IDA E VOLTA, entao:
          distancia (cm) = tempo (us) * 0,0343 / 2  =  tempo / 58,3
       E uma regra de tres: 58,3 us  <->  1 cm.
*/
const int TRIG = 9;
const int ECHO = 10;

void setup() {
  Serial.begin(9600);
  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);
  digitalWrite(TRIG, LOW);
}

void loop() {
  // 1) pulso de disparo: LOW limpo, depois HIGH por 10 us
  digitalWrite(TRIG, LOW);  delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG, LOW);

  // 2) mede quanto tempo o ECHO fica em HIGH (timeout de 30 ms = ~5 m)
  unsigned long tempo = pulseIn(ECHO, HIGH, 30000);
```

```
  // 3) converte tempo em distancia
  float distancia = tempo * 0.0343 / 2.0;     // cm

  Serial.print("tempo = ");
  Serial.print(tempo);
  Serial.print(" us   ->   distancia = ");
  if (tempo == 0) Serial.println("fora de alcance");
  else { Serial.print(distancia, 1); Serial.println(" cm"); }

  delay(100);   // o fabricante pede >= 60 ms entre medicoes (para o eco anterior "morrer")
}
```

Entenda o código

- `delayMicroseconds(10)` — o pulso de disparo tem que durar **pelo menos 10 µs**; `delay()` conta milissegundos, mil vezes mais.
- `pulseIn(ECHO, HIGH, 30000)` — espera o pino ir a HIGH, mede quanto tempo fica assim e devolve em µs. O terceiro argumento é o **timeout**: sem ele, o Arduino travaria se o eco nunca voltasse.
- `tempo * 0.0343 / 2.0` — a regra de três. `float` porque queremos casas decimais.
- `Serial.print(distancia, 1)` — imprime com 1 casa decimal.
- `delay(100)` — o fabricante pede ≥ 60 ms entre medições para o eco anterior “morrer”; senão uma medição contamina a próxima.

A porta serial: o Arduino conversando com o PC

Quando você abre o **Monitor Serial** (lupa, 9600 baud), cada `Serial.println()` vira uma linha de texto que viaja pelo cabo USB. **9600 baud** = 9600 bits por segundo ≈ 960 caracteres/s. É só isso: um cano de texto. Na Aula 2, o Python vai ler exatamente as mesmas linhas. Abra também o **Plotter Serial** (Ferramentas): ele desenha os números que chegam.

Experimente

1. Ponha um livro a 10, 20, 30 cm (meça com a régua). Anote o valor do sensor. Qual o erro em cm e em %?
2. Aponte para um travesseiro. Depois para uma parede inclinada. Explique o que acontece com o eco.
3. Troque `0.0343` por `0.0352` (som a 35 °C). Quanto muda a leitura a 100 cm?
4. Mude `delay(100)` para `delay(10)`. As leituras ficam estranhas? Por quê?

Código 1b — filtro de mediana e formato CSV

Leituras “loucas” aparecem de vez em quando (eco perdido, ruído elétrico). Um jeito robusto de ignorá-las: fazer 5 leituras, **ordenar** e pegar a do meio — a **mediana**. Uma média não resolveria: um valor absurdo (999) puxa a média para cima. Também mudamos o formato para `tempo,distância` (**CSV**), que o Plotter Serial e o Python entendem direto.

01b_hcsr04_filtro_csv.ino

```
/*
  01b - Leitura filtrada e saida em CSV (para o Plotter Serial e para o Python)
  Placa: Arduino Uno/Nano  -  mesmas ligacoes do sketch 01
```

```
  Duas melhorias:
    a) FILTRO DE MEDIANA: faz 5 leituras e usa a do meio. Uma leitura "louca"
       (eco perdido, ruido) nao passa. Media nao resolveria: um valor absurdo puxa a media.
    b) FORMATO: imprime "tempo_ms,distancia_cm" por linha. E um CSV - o Plotter Serial
       da IDE desenha na hora, e o Python le com um split(",").
*/
const int TRIG = 9, ECHO = 10;
const int N = 5;                      // leituras por medicao (impar!)

float lerUmaVez() {
  digitalWrite(TRIG, LOW);  delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG, LOW);
  unsigned long t = pulseIn(ECHO, HIGH, 30000);
  if (t == 0) return -1;              // sem eco
  return t * 0.0343 / 2.0;
}

float lerMediana() {
  float v[N];
  for (int i = 0; i < N; i++) { v[i] = lerUmaVez(); delay(15); }
  // ordena (bubble sort: pequeno e suficiente para 5 valores)
  for (int i = 0; i < N - 1; i++)
    for (int j = 0; j < N - 1 - i; j++)
      if (v[j] > v[j + 1]) { float tmp = v[j]; v[j] = v[j + 1]; v[j + 1] = tmp; }
  return v[N / 2];                    // o do meio
}

void setup() {
  Serial.begin(9600);
  pinMode(TRIG, OUTPUT); pinMode(ECHO, INPUT);
  Serial.println("tempo_ms,distancia_cm");   // cabecalho do CSV
}

void loop() {
  float d = lerMediana();
  Serial.print(millis());
  Serial.print(",");
  Serial.println(d, 1);               // -1.0 significa "fora de alcance"
}
```

- O `bubble sort` de 5 elementos: dois `for` que trocam vizinhos fora de ordem. Para 5 valores é mais que suficiente.
- `v[N / 2]` com N = 5 é `v[2]`: o terceiro dos cinco, o do meio.
- Teste: com o Plotter Serial aberto, passe a mão rápido na frente do sensor. Compare a curva do sketch 01 com a do 01b.

Desafios de lógica (Aula 1)

1. **1.1 Régua falante** — imprima também a distância em **polegadas** (1 pol = 2,54 cm) e em **milímetros**. Onde entra a regra de três?
2. **1.2 Máximo e mínimo** — guarde a menor e a maior distância vistas desde que ligou e imprima as duas a cada leitura. (Dica: duas variáveis e dois `if`.)
3. **1.3 Aproximando ou afastando?** — compare a leitura atual com a anterior e imprima `<<`, `>>` ou `==` (com uma tolerância de 1 cm para não “tremer”).
4. **1.4 Média móvel** — em vez da mediana, faça a média das últimas 8 leituras (vetor circular). Qual filtro reage mais rápido a uma mudança brusca? Qual é mais suave?
5. **1.5 Correção de temperatura** — receba a temperatura pelo Serial (`Serial.parseInt()`) e recalcule a velocidade do som com v = 331 + 0,6·T.

6. **1.6 Contador de passagem** — conte quantas vezes algo passou a menos de 30 cm (só conte na **borda**: quando entra, não enquanto está).

Aula 2 — Python lendo a serial e plotando ao vivo

O que você vai aprender

- Que a porta serial é a mesma para qualquer programa: IDE, Python, PuTTY…
- Ler linhas com **pyserial**, separar com `split(",")` e converter com `int()`/`float()`.
- Desenhar um gráfico que **se atualiza sozinho** com **matplotlib**.
- Gravar dados num CSV e abrir no Excel/LibreOffice.

Materiais

- A montagem da Aula 1 com o sketch **01b** gravado
- PC com Python 3 e as bibliotecas `pyserial` e `matplotlib`:

terminal

```
pip install pyserial matplotlib
# ou, no laboratório (pasta python-dados):  uv add pyserial matplotlib
```

> **Uma porta, um programa**
>
> Só **um** programa pode abrir a porta COM por vez. Antes de rodar o Python, **feche o Monitor Serial e o Plotter** da IDE. Se aparecer *Acesso negado / PermissionError*, é isso.

Passo 1 — só ler

ler_serial.py

```
"""Passo 1 em Python: so LER a porta serial e imprimir (antes de plotar).


Uso:  python ler_serial.py COM3
Mostra que a porta serial e apenas um "cano de texto": o que o Arduino escreve
com Serial.println() chega aqui linha por linha.
"""
import sys, time
import serial                                   # pip install pyserial
from serial.tools import list_ports


if len(sys.argv) < 2:
    print("Portas disponiveis:")
    for p in list_ports.comports(): print("  ", p.device, "-", p.description)
    print("uso: python ler_serial.py COM3"); sys.exit()


porta = sys.argv[1]
with serial.Serial(porta, 9600, timeout=1) as ser:
    time.sleep(2)                               # o Arduino reinicia quando a porta abre
    print("lendo", porta, "(Ctrl+C para parar)")
    while True:
        linha = ser.readline().decode("ascii", errors="ignore").strip()
        if not linha: continue
        print(repr(linha))                      # repr mostra exatamente o que chegou
        partes = linha.split(",")
        if len(partes) == 2 and partes[0].isdigit():
            t_ms, d_cm = int(partes[0]), float(partes[1])
            print(f"   -> t = {t_ms/1000:.1f} s   d = {d_cm} cm")
```

- `serial.Serial(porta, 9600, timeout=1)` — abre o cano; o baud tem que ser **igual** ao do sketch.

- `time.sleep(2)` — o Uno **reinicia** quando a porta abre (é o auto-reset); esperamos ele acordar.
- `readline()` devolve **bytes**; `.decode()` vira texto; `.strip()` tira o `\r\n` do fim.
- `repr(linha)` mostra a linha "crua" — ótimo para descobrir por que um `split` falhou.
- `partes[0].isdigit()` — só converte se a linha for mesmo `número,número` (a linha de cabeçalho não é).

Passo 2 — gráfico ao vivo

plot_distancia.py

```
"""Grafico AO VIVO de distancia x tempo lendo a porta serial do Arduino (sketch 01b).

Uso:
    python plot_distancia.py COM3            # porta real (9600 baud)
    python plot_distancia.py --simular       # sem Arduino: gera dados falsos para testar
    python plot_distancia.py COM3 --salvar leituras.csv   # tambem grava um CSV

Cada linha que o Arduino manda tem o formato:  tempo_ms,distancia_cm
Bibliotecas: pyserial, matplotlib   (pip install pyserial matplotlib)
"""
import sys, time, math, random, collections
import matplotlib
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation

JANELA_S = 30          # quantos segundos aparecem na tela
LIMITE_CM = 40         # linha vermelha de alarme (so visual)

# ---------- fonte de dados: serial real ou simulador ----------
class Serial_Arduino:
    def __init__(self, porta, baud=9600):
        import serial                       # pyserial
        self.ser = serial.Serial(porta, baud, timeout=1)
        time.sleep(2)                       # o Arduino reinicia ao abrir a porta
        self.ser.reset_input_buffer()
    def ler(self):
        """devolve (tempo_ms, distancia_cm) ou None"""
        linha = self.ser.readline().decode("ascii", errors="ignore").strip()
        if not linha or linha.startswith("tempo"):   # vazio ou cabecalho
            return None
        try:
            t, d = linha.split(",")
            return int(t), float(d)
        except ValueError:                  # linha incompleta/corrompida
            return None

class Simulador:
    """finge um objeto que se aproxima e se afasta, com ruido e um eco perdido de vez em quando"""
    def __init__(self):
        self.t0 = time.time()
    def ler(self):
        time.sleep(0.1)
        t = time.time() - self.t0
        d = 80 + 60 * math.sin(t / 4) + random.gauss(0, 1.5)
        if random.random() < 0.03: d = -1              # "fora de alcance"
        return int(t * 1000), round(d, 1)

# ---------- programa ----------
def main():
    args = [a for a in sys.argv[1:] if not a.startswith("--")]
    simular = "--simular" in sys.argv
    salvar = sys.argv[sys.argv.index("--salvar") + 1] if "--salvar" in sys.argv else None
    fonte = Simulador() if simular else Serial_Arduino(args[0] if args else "COM3")
    arquivo = open(salvar, "w", encoding="utf8") if salvar else None
    if arquivo: arquivo.write("tempo_s,distancia_cm\n")

    tempos = collections.deque()
    dists = collections.deque()
```

```
    t_inicio = None

    fig, ax = plt.subplots(figsize=(9, 4.5))
    fig.canvas.manager.set_window_title("Distancia x tempo - HC-SR04")
    linha, = ax.plot([], [], lw=2, color="#1f77b4", label="distância")
    ax.axhline(LIMITE_CM, color="red", ls="--", label=f"limite {LIMITE_CM} cm")
    texto = ax.text(0.01, 0.95, "", transform=ax.transAxes, fontsize=14, va="top")
    ax.set_xlabel("tempo (s)"); ax.set_ylabel("distância (cm)")
    ax.set_ylim(0, 200); ax.grid(alpha=0.3); ax.legend(loc="upper right")

    def atualiza(_frame):
        nonlocal t_inicio
        # le tudo o que chegou desde o ultimo quadro
        for _ in range(20):
            leitura = fonte.ler()
            if leitura is None: break
            t_ms, d = leitura
            if t_inicio is None: t_inicio = t_ms
            t_s = (t_ms - t_inicio) / 1000
            tempos.append(t_s); dists.append(d if d >= 0 else float("nan"))   # nan = buraco no grafico
            if arquivo: arquivo.write(f"{t_s:.2f},{d}\n")
        while tempos and tempos[-1] - tempos[0] > JANELA_S:
            tempos.popleft(); dists.popleft()
        if tempos:
            linha.set_data(tempos, dists)
            ax.set_xlim(max(0, tempos[-1] - JANELA_S), max(JANELA_S, tempos[-1]))
            ultimo = dists[-1]
            texto.set_text("fora de alcance" if ultimo != ultimo else f"{ultimo:.0f} cm")   # nan != nan
            texto.set_color("red" if ultimo == ultimo and ultimo < LIMITE_CM else "black")
        return linha, texto

    anim = FuncAnimation(fig, atualiza, interval=100, cache_frame_data=False)
    if "--png" in sys.argv:                                  # modo de teste: roda 8 s e salva imagem
        matplotlib.use("Agg")
        for i in range(80): atualiza(i)
        fig.savefig(sys.argv[sys.argv.index("--png") + 1], dpi=130, bbox_inches="tight")
        print("imagem salva")
        return
    plt.tight_layout(); plt.show()
    if arquivo: arquivo.close()

if __name__ == "__main__":
    main()
```

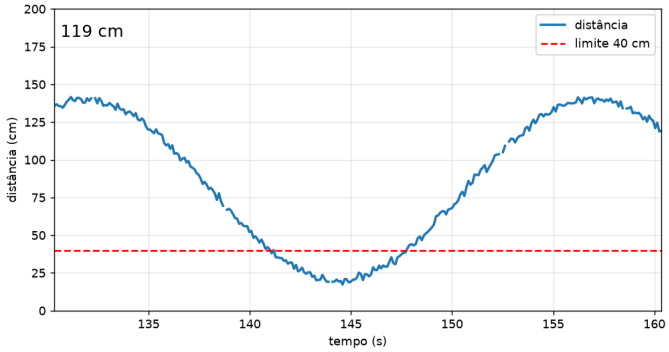


Figura 2 — Aula 2 — o gráfico ao vivo (aqui em modo `--simular`): a linha vermelha é o limite de alarme; o "buraco" na curva é uma leitura fora de alcance (NaN).

Entenda o código

- Duas **classes** com o mesmo método `ler()`: `Serial_Arduino` (porta real) e `Simulador` (dados falsos). O resto do programa não sabe a diferença — isso é **abstração**, e permite testar sem hardware.
- `deque` é uma fila: `append` põe no fim, `popleft` tira do começo. Mantemos só os últimos 30 s.

- `FuncAnimation` chama `atualiza()` a cada 100 ms; ela lê o que chegou e redesenha.
- `float("nan")` (not a number) faz o matplotlib **deixar um buraco** em vez de ligar pontos inválidos.
- `--salvar arquivo.csv` grava tudo: abra no LibreOffice Calc e faça o gráfico lá também.

Experimente

1. Rode `python plot_distancia.py --simular` para ver o gráfico sem o Arduino. Depois com a porta real (`COM3`, `COM4`…: `python ler_serial.py` sem argumentos lista as portas).
2. Mude `JANELA_S` para 10 e para 120.
3. Mude `LIMITE_CM` e observe o número ficar vermelho.
4. Grave 1 minuto em CSV com você andando na frente do sensor; abra no Calc e calcule média, mínimo e máximo.

Desafios de lógica (Aula 2)

1. **2.1 Velocidade** — a partir de duas leituras consecutivas (Δd/Δt), imprima a velocidade do objeto em cm/s. Faça um segundo gráfico com ela.
2. **2.2 Alarme no PC** — quando a distância ficar abaixo do limite, o Python toca um som (`print("\a")` ou `winsound.Beep`) e escreve no terminal a hora (`datetime.now()`).
3. **2.3 Estatística ao vivo** — mostre no título do gráfico a média e o desvio-padrão dos últimos 50 pontos (`statistics.mean`, `statistics.stdev`).
4. **2.4 Dois sentidos** — mande um comando do Python para o Arduino (`ser.write(b"L")`) que acende o LED do pino 13 quando o alarme dispara no PC. (O Arduino precisa de `Serial.read()`.)
5. **2.5 Detector de gestos** — “aproximar rápido e afastar” conta como um gesto; conte gestos por minuto. Pense nos estados antes de codar.

Aula 3 — Buzzer: fazendo som

O que você vai aprender

- O que é som (vibração), frequência (Hz) e nota musical.
- Buzzer **ativo** × **passivo**, e por que só o passivo toca melodias.
- `tone()`/`noTone()` e como uma melodia vira **dois vetores** (notas e durações).
- Onde achar centenas de músicas prontas (repositório brasileiro).

A física: vibração, frequência e altura

Som é o ar vibrando. Se algo vibra **440 vezes por segundo (440 Hz)**, ouvimos a nota **Lá**. Dobrando a frequência (880 Hz) temos o mesmo Lá uma oitava acima. O buzzer passivo tem um disco **piezoelétrico**: quando recebe tensão ele se curva um pouquinho; ligando e desligando 440 vezes por segundo, ele empurra o ar 440 vezes por segundo. É isso que `tone(pino, 440)` faz por baixo dos panos — um PWM cuja **frequência** você escolhe.

Tipo	Como reconhecer	Como usar	Serve para
Buzzer **ativo**	Tem oscilador dentro; costuma vir com adesivo em cima; apita ao ligar direto em 5 V	`digitalWrite(HIGH/LOW)`	Alarmes de um tom só
Buzzer **passivo** (MH-FMD)	Vê-se a placa piezo verde/preta; ligado direto em 5 V não apita (só um “tec”)	`tone(pino, freq)`	Sirenes, melodias, bipes variados

Montagem

Módulo MH-FMD	Arduino
S (sinal)	pino 8
+ (VCC)	5V (se o módulo tiver este pino)
− (GND)	GND

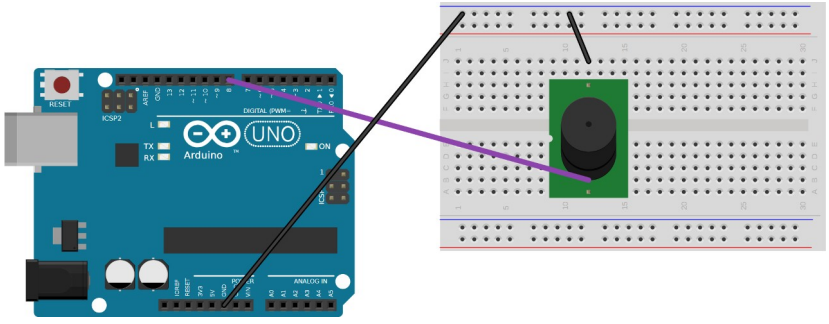


Figura 3 — Aula 3 — buzzer no pino 8 (o módulo MH-FMD tem um pino a mais, +, que vai ao 5V).

Código 2 — bipes, sirene e escala

```
02_buzzer_basico.ino
/*
  02 - Buzzer: fazendo som com o Arduino
```

```
  Placa: Arduino Uno/Nano

  Modulo MH-FMD (buzzer PASSIVO, 3 pinos):   S -> pino 8    + (VCC) -> 5V    - (GND) -> GND
  (Buzzer passivo sem modulo: uma perna no pino 8, outra no GND.)

  Fisica: som e vibracao do ar. O buzzer passivo tem um disco piezoeletrico que
  se deforma quando recebe tensao. Se ligarmos e desligarmos 440 vezes por segundo,
  ele vibra a 440 Hz = a nota LA. A funcao tone(pino, frequencia) faz exatamente isso.
  Um buzzer ATIVO ja tem o oscilador dentro: so liga/desliga (digitalWrite) e faz um
  unico apito fixo - nao serve para melodias.
*/
const int BUZZER = 8;

void setup() {
  pinMode(BUZZER, OUTPUT);
}

void loop() {
  // 1) tres bipes iguais
  for (int i = 0; i < 3; i++) {
    tone(BUZZER, 1000);      // 1000 Hz
    delay(150);
    noTone(BUZZER);
    delay(150);
  }
  delay(700);

  // 2) sirene: a frequencia sobe e desce
  for (int f = 400; f < 1200; f += 10) { tone(BUZZER, f); delay(5); }
  for (int f = 1200; f > 400; f -= 10) { tone(BUZZER, f); delay(5); }
  noTone(BUZZER);
  delay(700);

  // 3) escala: do (262) re (294) mi (330) fa (349) sol (392) la (440) si (494) do (523)
  int notas[] = { 262, 294, 330, 349, 392, 440, 494, 523 };
  for (int i = 0; i < 8; i++) { tone(BUZZER, notas[i], 200); delay(250); }   // tone com duracao
  delay(1500);
}
```

- `tone(pino, freq)` toca até `noTone()`; `tone(pino, freq, ms)` toca por um tempo e para sozinho (mas **não** espera: por isso o `delay(250)` depois).
- A sirene é um `for` na **frequência** — o mesmo truque de um `for` no brilho, só que no som.
- As notas da escala: cada uma é 2^(1/12) ≈ 1,0595 vezes a anterior (12 semitons dobram a frequência).

Código 2b — uma melodia

02b_melodia.ino

```
/*
  02b - Melodia: "Ode a Alegria" (Beethoven) no buzzer
  Placa: Arduino Uno/Nano    Buzzer passivo no pino 8

  Estrutura usada por quase todos os projetos de musica em Arduino
  (inclusive o repositorio brasileiro github.com/robsoncouto/arduino-songs):
    - constantes NOTE_xx com a frequencia de cada nota
    - vetor melodia[] com pares (nota, duracao): 4 = seminima, 8 = colcheia,
      numero negativo = nota pontuada (dura 1,5x)
    - um "tempo" em BPM que define quanto dura a seminima

  Troque o vetor melodia[] por qualquer musica do repositorio acima.
*/
#define NOTE_C4  262
#define NOTE_D4  294
#define NOTE_E4  330
```

```
#define NOTE_F4  349
#define NOTE_G4  392
#define NOTE_A4  440
#define NOTE_B4  494
#define NOTE_C5  523
#define REST       0

const int BUZZER = 8;
const int TEMPO = 114;                        // batidas por minuto

// nota, duracao  (4 = seminima, 8 = colcheia, -4 = seminima pontuada, 2 = minima)
int melodia[] = {
  NOTE_E4, 4, NOTE_E4, 4, NOTE_F4, 4, NOTE_G4, 4,
  NOTE_G4, 4, NOTE_F4, 4, NOTE_E4, 4, NOTE_D4, 4,
  NOTE_C4, 4, NOTE_C4, 4, NOTE_D4, 4, NOTE_E4, 4,
  NOTE_E4, -4, NOTE_D4, 8, NOTE_D4, 2,
  NOTE_E4, 4, NOTE_E4, 4, NOTE_F4, 4, NOTE_G4, 4,
  NOTE_G4, 4, NOTE_F4, 4, NOTE_E4, 4, NOTE_D4, 4,
  NOTE_C4, 4, NOTE_C4, 4, NOTE_D4, 4, NOTE_E4, 4,
  NOTE_D4, -4, NOTE_C4, 8, NOTE_C4, 2,
};

const int numNotas = sizeof(melodia) / sizeof(melodia[0]) / 2;
const int seminima = (60000 * 4) / TEMPO / 4;   // duracao da seminima em ms

void setup() {
  pinMode(BUZZER, OUTPUT);
  for (int i = 0; i < numNotas * 2; i += 2) {
    int nota = melodia[i];
    int divisor = melodia[i + 1];
    int duracao;
    if (divisor > 0) duracao = (seminima * 4) / divisor;
    else             duracao = (seminima * 4) / (-divisor) * 1.5;   // pontuada
    tone(BUZZER, nota, duracao * 0.9);   // toca 90% e deixa 10% de silencio: separa as notas
    delay(duracao);
    noTone(BUZZER);
  }
}

void loop() { }   // toca uma vez; aperte RESET para repetir
```

Entenda o código

- `#define NOTE_C4 262` — dá nome às frequências; C4 = Dó central.
- O vetor `melodia[]` alterna **nota, duração**: 4 = semínima, 8 = colcheia, 2 = mínima; **negativo** = pontuada (1,5×).
- `seminima = 60000 / TEMPO` ms — em 114 BPM cada batida dura 526 ms. Mudar `TEMPO` muda a velocidade sem mexer na música.
- Tocamos 90 % da duração e deixamos 10 % de silêncio: sem isso, duas notas iguais seguidas viram uma só.

Centenas de músicas prontas

O brasileiro **Robson Couto** mantém o repositório **github.com/robsoncouto/arduino-songs** com dezenas de músicas nesse mesmo formato (Star Wars, Super Mario, Tetris, Nokia, Zelda, Pantera Cor-de-Rosa, Ode à Alegria…). Cada pasta tem um `.ino` pronto: copie o vetor `melody[]`, o `tempo` e as definições de notas. Ele só pede **crédito** ao usar em vídeos/tutoriais.

No repositório o buzzer fica no pino 11 — mude para o seu pino.

Desafios de lógica (Aula 3)

1. **3.1 Parabéns** — transcreva “Parabéns pra você” (Dó Dó Ré Dó Fá Mi…) para o vetor. Quais durações?
2. **3.2 Toque de celular** — três notas curtas que repetem 3 vezes e param; um botão “atende”.
3. **3.3 Teclado de 1 tecla** — um potenciômetro escolhe a nota (`map` para 200–2000 Hz) e um botão toca. Depois: quantize para as 8 notas da escala (dica: vetor + `map` para índice 0–7).
4. **3.4 Piano por Serial** — digite `a s d f g h j k` no Monitor Serial e cada tecla toca uma nota (`Serial.read()` + `switch`).
5. **3.5 Melodia sem travar** — reescreva o 02b com `millis()` para o Arduino continuar lendo um botão enquanto toca (será necessário na Aula 4!).

Aula 4 — Mini projeto: alarme de ré

O que você vai aprender

- Integrar sensor + LED RGB + buzzer num único programa **sem `delay()` longo**.
- Organizar regras em **tabelas** (vetores) em vez de `if` encadeados.
- Bipe cuja cadência depende da distância — igual ao de um caminhão dando ré.

Materiais

- Montagem da Aula 1 (HC-SR04)
- LED RGB cátodo comum + 3 resistores 220 Ω
- Buzzer MH-FMD

Montagem

Componente	Pino do Arduino	Observação
HC-SR04 TRIG / ECHO	9 / 10	VCC 5V, GND
Buzzer S	8	
RGB R / G / B	5 / 6 / 3 (PWM ~)	resistor 220 Ω em cada; comum no GND

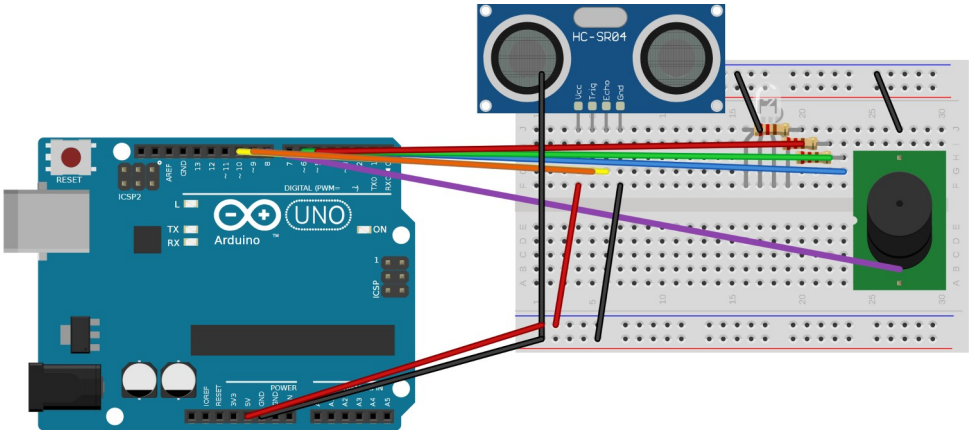


Figura 4 — Aula 4 — sensor, LED RGB e buzzer no Arduino Uno.

Código 3 — alarme de ré

03_alarme_re.ino

```
/*
  03 - Mini projeto: alarme de re (sensor de estacionamento)
  Placa: Arduino Uno/Nano

  Ligacoes:
    HC-SR04: VCC 5V, GND, TRIG 9, ECHO 10
    Buzzer (MH-FMD): S -> 8
    LED RGB catodo comum: R -> 220 -> 5(~)   G -> 220 -> 6(~)   B -> 220 -> 3(~)   comum -> GND

  Comportamento (igual ao de um caminhao):
    > 100 cm : verde, silencio
    60..100  : amarelo, bipe lento
    30..60   : laranja, bipe rapido
    < 30     : vermelho, bipe continuo
  Tudo SEM delay() longo: o bipe usa millis(), assim a distancia continua sendo lida
```

```
  enquanto o buzzer toca - o alarme reage na hora.
*/
const int TRIG = 9, ECHO = 10, BUZZER = 8;
const int PINO_R = 5, PINO_G = 6, PINO_B = 3;

// tabela de faixas: limite (cm), cor (r,g,b), intervalo do bipe (ms; 0 = continuo; -1 = silencio)
const int   LIMITE[]    = { 100, 60, 30, 0 };
const int   COR[][3]    = { {0,255,0}, {255,150,0}, {255,40,0}, {255,0,0} };
const int   INTERVALO[] = { -1, 600, 250, 0 };

float lerDistancia() {
  digitalWrite(TRIG, LOW);  delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG, LOW);
  unsigned long t = pulseIn(ECHO, HIGH, 30000);
  return (t == 0) ? 999 : t * 0.0343 / 2.0;    // sem eco = "muito longe"
}

int faixaDe(float d) {                          // devolve 0..3
  for (int i = 0; i < 3; i++) if (d > LIMITE[i]) return i;
  return 3;
}

void cor(int r, int g, int b) { analogWrite(PINO_R, r); analogWrite(PINO_G, g); analogWrite(PINO_B, b); }

void setup() {
  Serial.begin(9600);
  pinMode(TRIG, OUTPUT); pinMode(ECHO, INPUT); pinMode(BUZZER, OUTPUT);
  pinMode(PINO_R, OUTPUT); pinMode(PINO_G, OUTPUT); pinMode(PINO_B, OUTPUT);
}

unsigned long ultimaMedida = 0, ultimoBipe = 0;
bool bipando = false;
int faixa = 0;

void loop() {
  // 1) mede a cada 60 ms (nao mais rapido: o sensor precisa desse tempo)
  if (millis() - ultimaMedida >= 60) {
    ultimaMedida = millis();
    float d = lerDistancia();
    faixa = faixaDe(d);
    cor(COR[faixa][0], COR[faixa][1], COR[faixa][2]);
    Serial.print(d, 0); Serial.print(" cm  faixa "); Serial.println(faixa);
  }

  // 2) buzzer conforme a faixa, sem travar
  int intervalo = INTERVALO[faixa];
  if (intervalo < 0) { noTone(BUZZER); bipando = false; }            // silencio
  else if (intervalo == 0) { tone(BUZZER, 2000); }                    // continuo
  else if (millis() - ultimoBipe >= (unsigned long)intervalo) {       // bipe intermitente
    ultimoBipe = millis();
    bipando = !bipando;
    if (bipando) tone(BUZZER, 2000); else noTone(BUZZER);
  }
}
```

Entenda o código

- As **regras** estão em três vetores paralelos: `LIMITE[]`, `COR[][]`, `INTERVALO[]`. A faixa *i* vale acima de `LIMITE[i]`. Mudar o comportamento = mudar a tabela, não a lógica.
- `faixaDe(d)` devolve 0..3 — uma função que **classifica**. Todo o resto do programa só olha `faixa`.
- O bipe intermitente é um **toggle por tempo**: a cada `INTERVALO[faixa]` ms inverte `bipando`. `-1` = silêncio, `0` = contínuo. Um número codifica três comportamentos — e o `if/else if` os separa.

- Duas “tarefas” convivem no `loop()`: medir (a cada 60 ms) e bipar (a cada `intervalo` ms), cada uma com o seu `millis()`. É o padrão de multitarefa simples.

Experimente

1. Mude a tabela: 4 faixas → 6 faixas, mais suaves.
2. Troque a frequência do bipe por faixa (agudo quando perto).
3. Cubra o sensor com a mão: qual faixa aparece? Por quê (`999`)?

Desafios de lógica (Aula 4)

1. **4.1 Sensor de vaga** — o alarme só ativa quando algo **fica** a menos de 50 cm por mais de 2 s (evita disparar com alguém passando). Dica: guardar quando entrou na faixa.
2. **4.2 Histerese** — um objeto parado exatamente a 60 cm faz o alarme alternar amarelo/laranja sem parar. Resolva: só muda de faixa se passar do limite em **2 cm**.
3. **4.3 Melodia de alerta** — em vez de bipes, toque um trecho do 02b quando entrar na faixa vermelha, **sem travar** a leitura (é o desafio 3.5 aplicado).
4. **4.4 Silenciar** — um botão silencia o buzzer por 30 s (o LED continua). Depois de 30 s volta sozinho.
5. **4.5 Calibrar pelo pot** — um potenciômetro ajusta o limite da faixa vermelha entre 10 e 100 cm (e imprime no Serial).
6. **4.6 Registrar no PC** — mande `distância,faixa` pela serial e adapte o `plot_distancia.py` para pintar o fundo com a cor da faixa.

Aula 5 — ESP32: monitor web no celular

O que você vai aprender

- Migrar o alarme para o ESP32 — e por que o ECHO precisa de um **divisor de tensão**.
- Servir uma página que **pergunta ao ESP32** a distância a cada 300 ms (`fetch` + JSON).
- Desenhar um gráfico ao vivo no navegador (`<canvas>`), sem biblioteca nenhuma.
- Configurar o alarme pela própria página.

A física: 5 V encontra 3,3 V

O HC-SR04 é alimentado com 5 V e o pino ECHO **devolve 5 V**. As entradas do ESP32 aguentam no máximo **3,3 V**. A solução é um **divisor de tensão** com dois resistores: 1 kΩ e 2 kΩ em série entre ECHO e GND. No ponto entre eles a tensão é 5 V × 2 kΩ ÷ (1 kΩ + 2 kΩ) = **3,33 V**. É outra regra de três — desta vez de tensões, proporcional às resistências. O TRIG não precisa: o sensor aceita 3,3 V como “nível alto”.

Materiais

- ESP32 Dev Module + protoboard 830
- HC-SR04, buzzer, LED RGB + 3 × 220 Ω
- 1 resistor de 1 kΩ e 1 de 2 kΩ (ou 2,2 kΩ)
- Celular com Wi-Fi

Montagem

Componente	ESP32	Observação
HC-SR04 VCC / GND	VIN (5 V) / GND	VIN = 5 V do USB
TRIG	GPIO 5	direto
ECHO	1 kΩ → **GPIO 18** → 2 kΩ → GND	divisor: 3,3 V no GPIO
Buzzer S	GPIO 19	
RGB R / G / B	GPIO 25 / 26 / 27	220 Ω; comum no GND

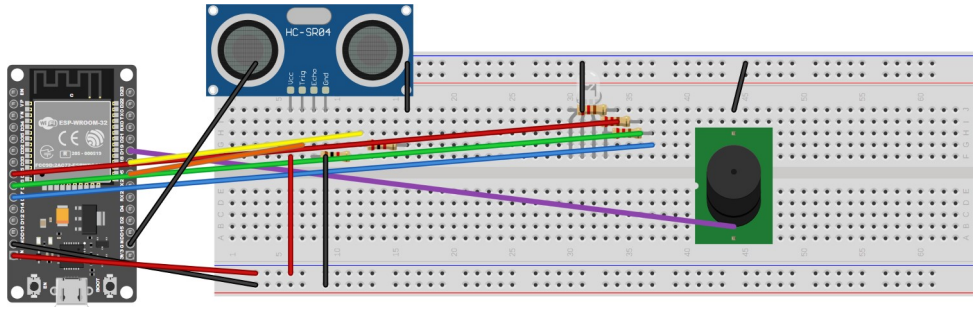


Figura 5 — Aulas 5 e 6 — ESP32 com o divisor 1 kΩ/2 kΩ no ECHO, LED RGB e buzzer. O sensor recebe 5 V do pino VIN.

Código 4 — alarme com monitor web

04_esp32_alarme_web.ino

```
/*
  04 - Alarme de distancia no ESP32 com monitor web local
```

```
  Placa: ESP32 Dev Module (core 3.x)

  Ligacoes:
    HC-SR04: VCC -> VIN (5 V)   GND -> GND   TRIG -> GPIO 5
             ECHO -> divisor de tensao -> GPIO 18
             ATENCAO: o ECHO sai com 5 V e o ESP32 so aguenta 3,3 V nas entradas!
             Divisor: ECHO -> resistor 1 kOhm -> GPIO 18 -> resistor 2 kOhm -> GND  (5 V * 2/3 = 3,3 V)
    Buzzer (MH-FMD): S -> GPIO 19
    LED RGB: R -> 220 -> GPIO 25   G -> 220 -> GPIO 26   B -> 220 -> GPIO 27   comum -> GND

  O ESP32 cria a rede "Alarme-ESP32" (senha 12345678). No celular, abra http://192.168.4.1
  A pagina pede /dados a cada 300 ms e desenha o grafico distancia x tempo. O limite de alarme
  pode ser mudado pela pagina (rota /limite?cm=40).
*/
#include <WiFi.h>
#include <WebServer.h>

const int TRIG = 5, ECHO = 18, BUZZER = 19;
const int PINO_R = 25, PINO_G = 26, PINO_B = 27;
const char* SSID_AP = "Alarme-ESP32";
const char* SENHA_AP = "12345678";

WebServer servidor(80);

// ---------- estado ----------
float distancia = 999;
int limiteAlarme = 40;          // cm: abaixo disso = ALARME
bool emAlarme = false;
unsigned long ultimaMedida = 0, ultimoBipe = 0, inicioAlarme = 0;
int totalAlarmes = 0;
bool bipando = false;

const char PAGINA[] PROGMEM = R"HTML(
<!DOCTYPE html><html lang="pt-br"><head><meta charset="utf-8"><meta name="viewport" content="width=device-
width,initial-scale=1">
<title>Alarme de distancia</title>
<style>
 body{font-family:sans-serif;background:#111;color:#eee;margin:0;padding:14px;text-align:center}
 .card{background:#222;border-radius:12px;padding:14px;margin:10px auto;max-width:520px}
 #dist{font-size:3em;font-weight:bold} #alarme{font-size:1.4em;padding:8px;border-radius:8px}
 .ok{background:#2a2} .al{background:#c22;animation:p 1s infinite} @keyframes p{50%{opacity:.4}}
 canvas{width:100%;height:220px;background:#000;border-radius:8px}
 input{width:70px;font-size:1.1em} button{font-size:1em;padding:8px 14px;border:0;border-
radius:8px;background:#448;color:#fff}
</style></head><body>
<h2>Sensor HC-SR04 — monitor</h2>
<div class="card"><div id="dist">-- cm</div><div id="alarme" class="ok">normal</div>
 <p>alarmes desde que ligou: <b id="n">0</b></p></div>
<div class="card"><canvas id="g" width="500" height="220"></canvas><p>últimos 60 s</p></div>
<div class="card">limite de alarme: <input type="number" id="lim" value="%LIMITE%" min="5" max="300"> cm
 <button onclick="fetch('/limite?cm='+document.getElementById('lim').value)">aplicar</button></div>
<script>
 const hist=[]; const c=document.getElementById('g').getContext('2d');
 function desenha(lim){const W=500,H=220,max=200;c.clearRect(0,0,W,H);
   c.strokeStyle='#c22';c.beginPath();const yl=H-lim/max*H;c.moveTo(0,yl);c.lineTo(W,yl);c.stroke();
   c.strokeStyle='#4af';c.lineWidth=2;c.beginPath();
   hist.forEach((v,i)=>{const x=i/200*W,y=H-Math.min(v,max)/max*H;i?
c.lineTo(x,y):c.moveTo(x,y)});c.stroke();c.lineWidth=1;
   c.fillStyle='#888';c.font='12px sans-serif';c.fillText('0 cm',4,H-4);c.fillText(max+' cm',4,12);}
 async function tick(){try{const d=await (await fetch('/dados')).json();
   document.getElementById('dist').textContent=d.cm<0?'-- cm':d.cm.toFixed(0)+' cm';
   const a=document.getElementById('alarme');a.textContent=d.alarme?'ALARME! objeto a '+d.cm.toFixed(0)+'
cm':'normal';a.className=d.alarme?'al':'ok';
   document.getElementById('n').textContent=d.total;
   hist.push(d.cm<0?200:d.cm);if(hist.length>200)hist.shift();desenha(d.limite);}catch(e){}}
 setInterval(tick,300);
</script></body></html>
)HTML";

// ---------- sensor ----------
float lerDistancia() {
  digitalWrite(TRIG, LOW);  delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10);
```

```
  digitalWrite(TRIG, LOW);
  unsigned long t = pulseIn(ECHO, HIGH, 30000);
  return (t == 0) ? -1 : t * 0.0343 / 2.0;
}
void cor(int r, int g, int b) { ledcWrite(PINO_R, r); ledcWrite(PINO_G, g); ledcWrite(PINO_B, b); }

// ---------- rotas ----------
void paginaInicial() {
  String html = FPSTR(PAGINA);
  html.replace("%LIMITE%", String(limiteAlarme));
  servidor.send(200, "text/html; charset=utf-8", html);
}
void rotaDados() {      // JSON: a pagina (ou qualquer programa) le isto
  String json = "{\"cm\":" + String(distancia, 1) + ",\"alarme\":" + (emAlarme ? "true" : "false") +
                ",\"limite\":" + String(limiteAlarme) + ",\"total\":" + String(totalAlarmes) + "}";
  servidor.send(200, "application/json", json);
}
void rotaLimite() {
  int novo = servidor.arg("cm").toInt();
  if (novo >= 5 && novo <= 300) limiteAlarme = novo;
  servidor.send(200, "text/plain", String(limiteAlarme));
}

void setup() {
  Serial.begin(115200);
  pinMode(TRIG, OUTPUT); pinMode(ECHO, INPUT); pinMode(BUZZER, OUTPUT);
  ledcAttach(PINO_R, 5000, 8); ledcAttach(PINO_G, 5000, 8); ledcAttach(PINO_B, 5000, 8);
  WiFi.softAP(SSID_AP, SENHA_AP);
  Serial.print("Rede: "); Serial.print(SSID_AP); Serial.print("  ->  http://");
Serial.println(WiFi.softAPIP());
  servidor.on("/", paginaInicial);
  servidor.on("/dados", rotaDados);
  servidor.on("/limite", rotaLimite);
  servidor.begin();
}

void loop() {
  servidor.handleClient();

  if (millis() - ultimaMedida >= 60) {
    ultimaMedida = millis();
    distancia = lerDistancia();
    bool agora = (distancia > 0 && distancia < limiteAlarme);
    if (agora && !emAlarme) { totalAlarmes++; inicioAlarme = millis(); Serial.printf("ALARME #%d: %.0f
cm\n", totalAlarmes, distancia); }
    emAlarme = agora;
    if (emAlarme) cor(255, 0, 0);
    else if (distancia > 0 && distancia < limiteAlarme * 2) cor(255, 120, 0);
    else cor(0, 255, 0);
  }

  // buzzer: bipe cuja rapidez depende de quao perto esta
  if (emAlarme) {
    int intervalo = map((int)distancia, 0, limiteAlarme, 60, 400);
    if (millis() - ultimoBipe >= (unsigned long)intervalo) {
      ultimoBipe = millis(); bipando = !bipando;
      if (bipando) tone(BUZZER, 2200); else noTone(BUZZER);
    }
  } else { noTone(BUZZER); bipando = false; }
}
```

Entenda o código

- Três rotas: `/` (a página), `/dados` (responde **JSON**: `{"cm":37.5,"alarme":true,...}`) e `/limite?cm=40` (muda o limite).
- JSON é texto num formato que o JavaScript lê direto: `d.cm`, `d.alarme`. É o “CSV da web”.

- A página faz `fetch('/dados')` a cada 300 ms (`setInterval`) — **polling**. Guarda os últimos 200 pontos num vetor `hist` e desenha no `<canvas>` com linhas: é o `plot_distancia.py` reescrito em JavaScript.
- `totalAlarmes++` só acontece na **borda** (`agora && !emAlarme`) — a mesma lógica de contar passagens da Aula 1.
- `map(distancia, 0, limite, 60, 400)` faz o bipe acelerar conforme aproxima: 400 ms longe, 60 ms colado.

Como testar

1. Grave, abra o Monitor Serial (115200): aparece `Rede: Alarme-ESP32 -> http://192.168.4.1`.
2. No celular, conecte-se à rede **Alarme-ESP32** (senha `12345678`). Desligue os dados móveis se a página não abrir.
3. Abra `http://192.168.4.1`. Passe a mão na frente do sensor e veja o gráfico e o alarme.
4. Mude o limite pela página e confira o LED.

Desafios de lógica (Aula 5)

1. **5.1 Histórico de alarmes** — guarde os instantes (`millis()`) dos últimos 10 alarmes e mostre numa lista na página (rota `/historico`).
2. **5.2 Silenciar pela web** — botão “silenciar 30 s” na página (rota `/silenciar`); o ESP32 respeita o tempo.
3. **5.3 Duas distâncias** — segundo HC-SR04 (GPIO 16/17 + divisor) e duas curvas no gráfico. Que cuidado é preciso para um sensor não ouvir o eco do outro?
4. **5.4 Modo estação** — troque `softAP` por `WiFi.begin` na rede da escola; descubra o IP pelo Serial. O que muda para o celular?
5. **5.5 Python de novo** — em vez do navegador, um script Python lê `http://192.168.4.1/dados` com `requests` a cada 0,3 s e plota com o mesmo `matplotlib` da Aula 2. Compare serial × Wi-Fi.

Aula 6 — Alerta pela internet: WhatsApp e e-mail

O que você vai aprender

- A diferença entre **rede local** (Aula 5) e **internet**: o ESP32 agora entra numa rede com saída para o mundo.
- O que é uma **API**: um endereço que, ao ser chamado, faz algo (mandar um WhatsApp).
- Como funciona o **SMTP**, o protocolo de e-mail de 1982 — conversando com o Gmail em texto puro.
- Por que existe **cooldown** e o que são credenciais.

Preparação (uma vez, com o professor)

1. **WhatsApp (CallMeBot, gratuito, uso pessoal):** salve o contato **+34 644 87 21 57**, envie pelo WhatsApp a mensagem `I allow callmebot to send me messages` e anote a **APIKEY** que chega em até 2 min. A API só manda mensagens **para o seu próprio número**.
2. **E-mail (Gmail):** na conta Google ative a *verificação em 2 etapas* e crie uma **Senha de app** (16 letras). Ela vai no código — nunca a senha normal.
3. Preencha no sketch 05: rede Wi-Fi com internet, número e APIKEY, e-mail e senha de app.

Código 5 — alerta por WhatsApp e e-mail

05_esp32_alerta_internet.ino

```
/*
  05 - Alerta pela INTERNET: WhatsApp (CallMeBot) e e-mail (SMTP do Gmail)
  Placa: ESP32 Dev Module (core 3.x)   Sem bibliotecas extras.
  Ligacoes: iguais ao sketch 04

  O e-mail e enviado "na mao", conversando com o servidor SMTP do Gmail em TEXTO PUTO:
  EHLO, AUTH LOGIN, MAIL FROM, RCPT TO, DATA, QUIT. Abra o Monitor Serial e veja o dialogo -
  e o mesmo protocolo de 1982 que todo programa de e-mail usa por baixo dos panos.
  (Alternativa com biblioteca: "ESP Mail Client", de Mobizt - precisa da particao "Huge APP".)

  Diferenca em relacao ao sketch 04: agora o ESP32 ENTRA na rede Wi-Fi da escola (que tem internet)
  em vez de criar a propria. Quando um objeto passa do limite, ele:
     1) manda uma mensagem de WhatsApp pela API gratuita CallMeBot
     2) manda um e-mail pelo servidor SMTP do Gmail
  Ha um "tempo de silencio" (COOLDOWN) para nao mandar 50 mensagens num unico evento.

  --- Preparacao (uma vez) ---
  WhatsApp: salve o contato +34 644 87 21 57, mande "I allow callmebot to send me messages" e
            anote a APIKEY que vem na resposta (uso pessoal; so manda para o SEU numero).
  E-mail:   na conta Google, ative a verificacao em 2 etapas e crie uma "Senha de app" (16 letras).
            Use essa senha aqui, NUNCA a senha normal.
*/
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include <base64.h>

// ---------- preencha ----------
const char* WIFI_SSID  = "NOME-DA-REDE";
const char* WIFI_SENHA = "SENHA-DA-REDE";
const char* ZAP_FONE   = "+5581999999999";     // seu numero com DDI+DDD
const char* ZAP_APIKEY = "123456";             // apikey do CallMeBot
const char* EMAIL_DE   = "seu.email@gmail.com";
const char* EMAIL_SENHA_APP = "abcd efgh ijkl mnop";
const char* EMAIL_PARA = "destino@exemplo.com";

// ---------- hardware ----------
const int TRIG = 5, ECHO = 18, BUZZER = 19;
```

```
const int PINO_R = 25, PINO_G = 26, PINO_B = 27;
const int LIMITE_CM = 40;
const unsigned long COOLDOWN_MS = 60000;       // no maximo 1 alerta por minuto

unsigned long ultimoAlerta = 0;
bool emAlarme = false;
int totalAlarmes = 0;

float lerDistancia() {
  digitalWrite(TRIG, LOW);  delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG, LOW);
  unsigned long t = pulseIn(ECHO, HIGH, 30000);
  return (t == 0) ? -1 : t * 0.0343 / 2.0;
}
void cor(int r, int g, int b) { ledcWrite(PINO_R, r); ledcWrite(PINO_G, g); ledcWrite(PINO_B, b); }

// ---------- WhatsApp ----------
String urlEncode(String s) {                    // espaco -> %20 etc.
  String out = "";
  const char* hex = "0123456789ABCDEF";
  for (unsigned int i = 0; i < s.length(); i++) {
    char c = s[i];
    if (isalnum(c) || c == '-' || c == '_' || c == '.' || c == '~') out += c;
    else { out += '%'; out += hex[(c >> 4) & 0xF]; out += hex[c & 0xF]; }
  }
  return out;
}
bool enviarWhatsApp(String texto) {
  WiFiClientSecure cliente;
  cliente.setInsecure();                        // nao valida o certificado (ok para a aula)
  HTTPClient http;
  String url = "https://api.callmebot.com/whatsapp.php?phone=" + String(ZAP_FONE) +
               "&text=" + urlEncode(texto) + "&apikey=" + String(ZAP_APIKEY);
  http.begin(cliente, url);
  int codigo = http.GET();
  http.end();
  Serial.printf("WhatsApp: HTTP %d\n", codigo);
  return codigo == 200;
}

// ---------- E-mail (SMTP "na mao") ----------
// manda uma linha ao servidor e espera a resposta; o SMTP responde com um codigo de 3 digitos
bool smtpFala(WiFiClientSecure& c, String linha, int esperado) {
  if (linha.length()) { c.println(linha); Serial.println("> " + (linha.startsWith("AUTH") ||
linha.length() == 24 ? "(credencial)" : linha)); }
  unsigned long t0 = millis();
  while (!c.available() && millis() - t0 < 8000) delay(10);
  String resp = c.readStringUntil('\n');
  Serial.println("< " + resp);
  return resp.startsWith(String(esperado));
}

bool enviarEmail(String assunto, String texto) {
  WiFiClientSecure c;
  c.setInsecure();
  if (!c.connect("smtp.gmail.com", 465)) { Serial.println("SMTP: nao conectou"); return false; }
  String senha = EMAIL_SENHA_APP; senha.replace(" ", "");            // a senha de app vem com espacos
  bool ok = smtpFala(c, "", 220)
         && smtpFala(c, "EHLO esp32", 250)
         && smtpFala(c, "AUTH LOGIN", 334)
         && smtpFala(c, base64::encode(EMAIL_DE), 334)
         && smtpFala(c, base64::encode(senha), 235)
         && smtpFala(c, "MAIL FROM:<" + String(EMAIL_DE) + ">", 250)
         && smtpFala(c, "RCPT TO:<" + String(EMAIL_PARA) + ">", 250)
         && smtpFala(c, "DATA", 354);
  if (ok) {
    c.println("From: Alarme ESP32 <" + String(EMAIL_DE) + ">");
    c.println("To: <" + String(EMAIL_PARA) + ">");
    c.println("Subject: " + assunto);
    c.println("Content-Type: text/plain; charset=utf-8");
    c.println();
    c.println(texto);
    ok = smtpFala(c, ".", 250);                                      // linha so com "." = fim da mensagem
```

```
  }
  smtpFala(c, "QUIT", 221);
  c.stop();
  Serial.println(ok ? "E-mail enviado!" : "E-mail: falhou (veja o dialogo acima)");
  return ok;
}

void setup() {
  Serial.begin(115200);
  pinMode(TRIG, OUTPUT); pinMode(ECHO, INPUT); pinMode(BUZZER, OUTPUT);
  ledcAttach(PINO_R, 5000, 8); ledcAttach(PINO_G, 5000, 8); ledcAttach(PINO_B, 5000, 8);

  WiFi.begin(WIFI_SSID, WIFI_SENHA);
  Serial.print("Conectando ao Wi-Fi");
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.print("\nConectado. IP: "); Serial.println(WiFi.localIP());
  cor(0, 255, 0);
}

void loop() {
  float d = lerDistancia();
  bool agora = (d > 0 && d < LIMITE_CM);

  if (agora && !emAlarme) {                                  // BORDA: acabou de entrar em alarme
    totalAlarmes++;
    Serial.printf("ALARME #%d: %.0f cm\n", totalAlarmes, d);
    cor(255, 0, 0); tone(BUZZER, 2200, 300);
    if (millis() - ultimoAlerta > COOLDOWN_MS || ultimoAlerta == 0) {
      ultimoAlerta = millis();
      String texto = "ALERTA: objeto detectado a " + String(d, 0) + " cm (alarme #" + String(totalAlarmes)
+ ")";
      cor(0, 0, 255);                                         // azul = enviando
      enviarWhatsApp(texto);                                  // ~2 s
      enviarEmail("Alarme ESP32 disparou", texto);            // ~5 s
      cor(255, 0, 0);
    } else Serial.println("(dentro do cooldown, nao envia de novo)");
  }
  if (!agora && emAlarme) cor(0, 255, 0);
  emAlarme = agora;
  delay(100);
}
```

Entenda o código

- **WhatsApp** é um único pedido HTTPS: `https://api.callmebot.com/whatsapp.php?phone=…&text=…&apikey=…`. `urlEncode` troca espaços e acentos por `%20`, `%C3%A7`… porque uma URL não pode ter espaços.
- `WiFiClientSecure` + `setInsecure()`: HTTPS criptografa a conversa, mas não estamos verificando o certificado do servidor (ok para a aula; num produto, verificaríamos).
- **E-mail**: `enviarEmail` conversa com `smtp.gmail.com` na porta 465. Veja no Serial: `220` (olá), `EHLO`, `AUTH LOGIN` + usuário e senha em **base64** (não é criptografia, só codificação — o sigilo vem do TLS), `MAIL FROM`, `RCPT TO`, `DATA`, a mensagem, um `.` sozinho e `QUIT`. Cada resposta tem um **código de 3 dígitos**; 2xx/3xx = ok, 5xx = erro.
- **Cooldown**: `COOLDOWN_MS = 60000` — no máximo 1 alerta por minuto. Sem isso, um cachorro parado na frente do sensor geraria centenas de mensagens.
- O LED fica **azul** enquanto envia (2–7 s): o `loop()` **trava** durante o envio. Aceitável aqui; o desafio 6.3 resolve.

Testando

1. Monitor Serial a 115200. Aproxime a mão a menos de 40 cm.
2. Deve aparecer `WhatsApp: HTTP 200` e o diálogo SMTP terminando em `E-mail enviado!`.
3. Confira o celular e a caixa de entrada (e o spam).
4. Aproxime de novo antes de 1 min: “dentro do cooldown”.

Desafios de lógica (Aula 6)

1. **6.1 Mensagem melhor** — inclua no texto a hora (peça a hora ao servidor NTP: `configTime(-3*3600, 0, "pool.ntp.org")` e `getLocalTime`) e o número do alarme do dia.
2. **6.2 Resumo diário** — em vez de um e-mail por alarme, um único e-mail às 18h com o total (usa a hora do desafio 6.1).
3. **6.3 Enviar sem travar** — mova o envio para uma **segunda tarefa** (`xTaskCreate`) no outro núcleo do ESP32, para o sensor e o buzzer não pararem enquanto envia.
4. **6.4 Confirmação** — o LED pisca verde 3 vezes se WhatsApp **e** e-mail deram certo; vermelho se algum falhou. (Dois `bool` e uma tabela-verdade.)
5. **6.5 Outro canal** — troque o WhatsApp por **Telegram** (bot + `api.telegram.org/bot<TOKEN>/sendMessage`) ou por **ntfy.sh** (um `POST` simples). Qual foi mais fácil? Por quê?
6. **6.6 Local + internet** — junte os sketches 04 e 05: monitor web local **e** alerta pela internet no mesmo programa (o ESP32 pode ser AP e estação ao mesmo tempo: `WIFI_AP_STA`).

Para ir além

- Sensor de distância a **laser** (VL53L0X, I²C): compare precisão e cone com o HC-SR04.
- Mandar as leituras por **MQTT** para o painel do laboratório (InfluxDB + Grafana) — um gráfico de semanas.
- Um **robô** que desvia de obstáculos com o HC-SR04 e dois motores.
- Contador de pessoas numa porta com dois sensores (direção de passagem).

Glossário

Termo	Significado
Ultrassom	Som acima de 20 kHz; o HC-SR04 usa 40 kHz.
Eco / tempo de voo	Tempo entre emitir e receber o som de volta; proporcional à distância (ida e volta).
pulseIn	Mede quanto tempo um pino fica em HIGH (ou LOW), em µs.
Mediana / média móvel	Filtros: o valor do meio de N leituras / a média das últimas N.
Serial / baud	Canal de texto entre placa e PC; baud = bits por segundo.
CSV / JSON	Formatos de texto para dados: `a,b,c` / `{"a":1}`.
Divisor de tensão	Dois resistores em série que reduzem uma tensão proporcionalmente.
Piezoelétrico	Material que se deforma com tensão elétrica (e vice-versa).
Hz / BPM	Ciclos por segundo (frequência) / batidas por minuto (andamento).
Histerese	Limites diferentes para entrar e sair de um estado, evitando oscilação.
Polling	Perguntar repetidamente (“já mudou?”) em vez de ser avisado.
API	Endereço/função que um programa chama para pedir um serviço (ex.: mandar mensagem).
HTTPS / TLS	HTTP com criptografia; protege senhas e mensagens no caminho.
SMTP	Protocolo de envio de e-mail (texto, códigos de 3 dígitos).
Base64	Codificação de bytes em texto; não é criptografia.
Cooldown	Tempo mínimo entre dois alertas.

Anexo — Ficha de medições (Aula 1)

Distância real (cm)	Tempo do eco (µs)	Distância medida (cm)	Erro (cm)	Erro (%)
5				
10				
20				
40				
80				
150				

Regra de três de conferência: tempo esperado = distância × 2 ÷ 0,0343. Para 20 cm: 20 × 2 ÷ 0,0343 ≈ **1166 µs**.

Anexo — Checklist

- [ ] HC-SR04: VCC 5V, GND, TRIG e ECHO nos pinos do código
- [ ] ESP32: divisor 1 kΩ/2 kΩ no ECHO; sensor no VIN
- [ ] Buzzer passivo (não ativo) para melodias
- [ ] LED RGB com 3 resistores; comum no GND
- [ ] Monitor Serial fechado antes de rodar o Python
- [ ] Baud igual no sketch e no Python (9600) / 115200 no ESP32
- [ ] Celular na rede do ESP32 com dados móveis desligados (Aula 5)
- [ ] Credenciais preenchidas e **não** compartilhadas (Aula 6)